NASCOMSOFT EMBEDDED LTD

WEEK 2 - DAY 2
BUILDING APIs PRACTICALLY WITH FASTAPI

Detailed Self-Learning Workbook | FastAPI First | CRUD Clearly Explained | Pydantic Introduced Last

Today's goal	Turn Day 1 HTTP/API theory into a running FastAPI application and understand CRUD in practice.

1. How to Use This Workbook

This workbook is written so that a student can follow the lesson independently. Do not jump straight to the final code. Type each stage, run it, test it, and only then continue.

1. Read the explanation before typing the code.
2. Type examples yourself rather than copying the final program immediately.
3. Run the server after each meaningful change.
4. Test endpoints in the browser or FastAPI interactive documentation.
5. Compare what you see with the expected result.
6. If something fails, use the troubleshooting section before moving on.

Important learning order
FastAPI basics come first. CRUD is then taught as a separate software-development concept and implemented with simple Python data structures. Pydantic is intentionally delayed until the final part of the workbook.

2. What You Are Building Today

You will build the first working version of a Task Management API. It will not use a database yet. Tasks will live temporarily in a normal Python list so the focus stays on FastAPI itself and on how HTTP operations become Python endpoint functions.

```
PYTHON

GET    /tasks
GET    /tasks/{task_id}
POST   /tasks
PUT    /tasks/{task_id}
```

```
DELETE /tasks/{task_id}
```

3. FastAPI: The Core Idea

FastAPI is a Python web framework for building APIs. The most important beginner idea is simple: FastAPI connects an HTTP method and URL path to a Python function.

PYTHON

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello from FastAPI"}
```

Line / Code	What it does	Why it matters
from fastapi import FastAPI	Imports the FastAPI class.	Python must import the framework class before it can be used.
app = FastAPI()	Creates the FastAPI application object.	This central object stores routes and application configuration.
@app.get("/")	Registers GET / for the next function.	This is the bridge between HTTP and Python.
def home():	Defines the function that handles the request.	Normal Python functions become endpoint handlers.
return {"message": ...}	Returns a Python dictionary.	FastAPI serializes it into JSON for the response.

Common mistake
@app.get("/") does not call home() immediately. It registers home() so FastAPI can call it later when a matching request arrives.

4. FastAPI and Uvicorn Are Different

FastAPI defines what the API should do. Uvicorn is the server program that listens for HTTP requests and runs the FastAPI application.

```
Client / Browser
    |
    | HTTP request
    v
 Uvicorn
    |
    v
 FastAPI
    |
    v
Python endpoint function
    |
```

```
  v
HTTP response
```

5. Project Setup - Step by Step

5.1 Create a project folder

```
TERMINAL

mkdir task_api
cd task_api
```

The first command creates the folder. The second command moves the terminal into it.

5.2 Create a virtual environment

```
TERMINAL

python -m venv .venv
```

The virtual environment isolates this project's packages from other Python projects.

5.3 Activate it

Windows PowerShell:

```
TERMINAL

.venv\Scripts\Activate.ps1
```

Windows Command Prompt:

```
TERMINAL

.venv\Scripts\activate
```

macOS/Linux:

```
TERMINAL

source .venv/bin/activate
```

5.4 Install FastAPI and Uvicorn

```
TERMINAL

pip install fastapi uvicorn
```

FastAPI is the framework. Uvicorn is the development server.

5.5 Create main.py

Create a file named main.py in the project folder.

6. Your First Running FastAPI Application

PYTHON

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Task Management API is running"}
```

Line / Code	What it does	Why it matters
from fastapi import FastAPI	Makes FastAPI available in this file.	Imports provide access to installed package functionality.
app = FastAPI()	Creates the app instance.	Uvicorn will later load this exact object.
@app.get("/")	Associates GET / with home().	The decorator registers the route.
def home():	Defines the endpoint function.	FastAPI runs it for matching requests.
return {...}	Produces response data.	FastAPI converts the dict to JSON.

6.1 Start the server

TERMINAL

```
uvicorn main:app --reload
```

Line / Code	What it does	Why it matters
uvicorn	Starts the Uvicorn server.	A server must listen for HTTP traffic.
main	Refers to main.py.	Python module names are written without .py.
app	Refers to the FastAPI object named app.	That is the application Uvicorn runs.
--reload	Restarts the server after source changes.	Useful during development.

Open http://127.0.0.1:8000. Expected response:

JSON

```
{
  "message": "Task Management API is running"
}
```

Checkpoint
If you can see that JSON, you have completed a real HTTP request-response cycle using your own FastAPI server.

7. Routes and Endpoints

A route connects an HTTP method and path to a Python function. An endpoint is the usable API operation created by that combination.

PYTHON

```
@app.get("/tasks")
def get_tasks():
    return []
```

Line / Code	What it does	Why it matters
@app.get	The method is GET.	The client is requesting data.
"/tasks"	The path is /tasks.	It represents the tasks resource.
get_tasks	The handler function.	It contains the endpoint logic.
return []	Returns an empty Python list.	FastAPI converts it to JSON [].

GET /tasks and POST /tasks are different endpoints because an endpoint is identified by both method and path.

8. FastAPI Interactive Documentation

Open http://127.0.0.1:8000/docs. FastAPI automatically generates interactive documentation for registered endpoints.

7. Expand an endpoint.
8. Click Try it out.
9. Enter any required value.
10. Click Execute.
11. Read the Request URL.
12. Read the response status.
13. Read the response body.

9. Add Temporary Task Data

```python
tasks = [
    {"id": 1, "title": "Learn FastAPI", "completed": False},
    {"id": 2, "title": "Build first API", "completed": False}
]
```

Line / Code	What it does	Why it matters
tasks = [...]	Creates a Python list.	It acts as temporary in-memory storage.
{"id": 1, ...}	Each task is a dictionary.	Dictionaries map naturally to JSON objects.
"completed": False	Stores a Boolean state.	FastAPI serializes False as JSON false.

Important limitation
The list is only stored in memory. Data created during the lesson disappears when the server process stops. Database persistence comes later.

10. CRUD - Understand the Concept Before Coding It

CRUD is a software-development concept describing the four basic operations performed on application data. It is not a FastAPI feature.

CRUD	Meaning	Typical HTTP method	Task example
Create	Add new data	POST	Create a task
Read	Retrieve data	GET	View tasks
Update	Change data	PUT / PATCH	Edit a task
Delete	Remove data	DELETE	Delete a task

CRUD describes what happens to data. HTTP methods describe how a client asks the API to perform those operations.

FLOW / ENDPOINT MAP

```
CREATE -> POST   /tasks
READ   -> GET    /tasks
READ   -> GET    /tasks/1
UPDATE -> PUT    /tasks/1
DELETE -> DELETE /tasks/1
```

Common mistake

CRUD and HTTP are related but not identical. CRUD is a data-operation vocabulary; GET/POST/PUT/PATCH/DELETE are HTTP methods.

11. READ - Get All Tasks

PYTHON

```python
@app.get("/tasks")
def get_tasks():
    return tasks
```

Line / Code	What it does	Why it matters
@app.get("/tasks")	Registers GET /tasks.	GET is appropriate because data is being read.
def get_tasks():	Defines the handler.	FastAPI calls it for GET /tasks.
return tasks	Returns the list.	FastAPI serializes it to a JSON array.

12. READ - Get One Task with a Path Parameter

PYTHON

```python
@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task
```

```
    return {"message": "Task not found"}
```

Line / Code	What it does	Why it matters
@app.get("/tasks/{task_id}")	Declares a variable path segment.	One route can handle /tasks/1, /tasks/2, etc.
task_id: int	Receives and validates the ID as an integer.	FastAPI converts URL text into Python int when possible.
for task in tasks:	Loops over stored tasks.	The code must search for the requested ID.
if task["id"] == task_id:	Checks for an ID match.	The match identifies the requested resource.
return task	Returns the matching dictionary.	It becomes JSON.

13. Query Parameters in FastAPI

PYTHON

```python
@app.get("/tasks")
def get_tasks(completed: bool | None = None):
    if completed is None:
        return tasks

    return [task for task in tasks if task["completed"] == completed]
```

Line / Code	What it does	Why it matters
completed: bool | None = None	Defines an optional query parameter.	It is not in the route path, so FastAPI treats it as a query value.
if completed is None	Checks whether filtering was omitted.	GET /tasks should still return all tasks.
return [task for ...]	Filters by completion state.	Supports ?completed=true or ?completed=false.

14. CREATE - POST a New Task Without Pydantic

For now, accept the request body as a plain Python dictionary. This keeps the focus on FastAPI and CRUD before adding validation models.

PYTHON

```python
from fastapi import FastAPI, Body

@app.post("/tasks", status_code=201)
def create_task(task: dict = Body(...)):
    new_task = {
        "id": len(tasks) + 1,
        "title": task.get("title", "Untitled task"),
        "completed": False
    }

    tasks.append(new_task)
    return new_task
```

Line / Code	What it does	Why it matters
Body	Imports FastAPI's Body helper.	It marks a function value as coming from the HTTP body.
@app.post(..., status_code=201)	Registers POST /tasks and sets 201 on success.	POST represents creation; 201 means created.
task: dict = Body(...)	Reads JSON as a Python dictionary.	This is intentionally simple before Pydantic.
len(tasks) + 1	Creates a demonstration ID.	Fine for a classroom list, not robust database ID generation.
task.get("title", ...)	Reads title from the dictionary.	dict.get avoids KeyError and allows a fallback.
"completed": False	Sets the initial task state.	This is an application rule.
tasks.append(new_task)	Adds the new dictionary to storage.	This is the CREATE operation.
return new_task	Returns what was created.	The client sees the resulting resource.

Test body:

```JSON
{
  "title": "Complete FastAPI practical"
}
```

15. UPDATE - PUT an Existing Task

```PYTHON
@app.put("/tasks/{task_id}")
def update_task(task_id: int, task_data: dict = Body(...)):
    for task in tasks:
        if task["id"] == task_id:
            task["title"] = task_data.get("title", task["title"])
            task["completed"] = task_data.get("completed", task["completed"])
            return task

    return {"message": "Task not found"}
```

Line / Code	What it does	Why it matters
@app.put("/tasks/{task_id}")	Registers a PUT endpoint for one ID.	The method expresses an update operation.
task_id: int	Gets the resource ID from the path.	The URL identifies which task to change.
task_data: dict = Body(...)	Reads update values from JSON.	The body carries new data.
for task in tasks	Searches for the target.	The resource must be found before it can be changed.
task["title"] = ...	Updates title when provided.	Keeps old value when omitted.
task["completed"] = ...	Updates completion state.	Again preserves old value if omitted.
return task	Returns the updated resource.	The client can verify the new state.

Teaching note
Strict REST discussions often distinguish PUT as full replacement and PATCH as partial modification. Today we keep the code beginner-friendly so the core routing, body and CRUD flow is clear.

16. DELETE - Remove a Task

PYTHON

```python
@app.delete("/tasks/{task_id}")
def delete_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            tasks.remove(task)
            return {"message": "Task deleted successfully"}

    return {"message": "Task not found"}
```

Line / Code	What it does	Why it matters
@app.delete("/tasks/{task_id}")	Registers DELETE for one task.	The method communicates removal intent.
for task in tasks	Searches for the resource.	We need the actual dictionary before removal.
tasks.remove(task)	Removes it from the list.	This is the DELETE data operation.
return {"message": ...}	Returns confirmation.	The client receives success feedback.

17. Proper 404 Error Responses

Returning an error-looking dictionary with HTTP 200 is misleading. Use HTTPException so the status code also communicates failure.

PYTHON

```python
from fastapi import FastAPI, Body, HTTPException

@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task

    raise HTTPException(
        status_code=404,
        detail="Task not found"
    )
```

Line / Code	What it does	Why it matters
HTTPException	Imports FastAPI's HTTP error class.	It lets the API return a real error status.
raise HTTPException(...)	Stops normal endpoint flow.	The response becomes the error immediately.
status_code=404	Sets Not Found.	Clients can distinguish missing data programmatically.
detail="Task not found"	Provides a readable explanation.	FastAPI returns it in the JSON detail field.

18. Complete CRUD Request Lifecycle

FLOW / ENDPOINT MAP

```
CLIENT
  |
  | POST /tasks + JSON body
 v
UVICORN
  |
 v
FASTAPI matches @app.post("/tasks")
  |
 v
create_task() receives Python dict
  |
 v
Python creates and appends new_task
  |
 v
FastAPI converts return value to JSON
  |
 v
HTTP 201 response returns to client
```

19. Complete FastAPI + CRUD Program Before Pydantic

PYTHON

```python
from fastapi import FastAPI, Body, HTTPException

app = FastAPI(title="Task Management API")

tasks = [
    {"id": 1, "title": "Learn FastAPI", "completed": False},
    {"id": 2, "title": "Build first API", "completed": False}
]

@app.get("/")
def home():
    return {"message": "Task Management API is running"}

@app.get("/tasks")
def get_tasks(completed: bool | None = None):
    if completed is None:
        return tasks
    return [task for task in tasks if task["completed"] == completed]

@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            return task
    raise HTTPException(status_code=404, detail="Task not found")
```

```
@app.post("/tasks", status_code=201)
def create_task(task: dict = Body(...)):
    new_task = {
        "id": len(tasks) + 1,
        "title": task.get("title", "Untitled task"),
        "completed": False
    }
    tasks.append(new_task)
    return new_task

@app.put("/tasks/{task_id}")
def update_task(task_id: int, task_data: dict = Body(...)):
    for task in tasks:
        if task["id"] == task_id:
            task["title"] = task_data.get("title", task["title"])
            task["completed"] = task_data.get("completed", task["completed"])
            return task
    raise HTTPException(status_code=404, detail="Task not found")

@app.delete("/tasks/{task_id}")
def delete_task(task_id: int):
    for task in tasks:
        if task["id"] == task_id:
            tasks.remove(task)
            return {"message": "Task deleted successfully"}
    raise HTTPException(status_code=404, detail="Task not found")
```

At this stage, students should understand the FastAPI application without Pydantic. That is intentional.

20. Systematic Testing Checklist

Test	Request	Expected status	Confirm
Root	GET /	200	Welcome JSON
Read all	GET /tasks	200	Task array
Read one	GET /tasks/1	200	Task 1
Missing	GET /tasks/999	404	Not found detail
Filter	GET /tasks?completed=false	200	Incomplete only
Create	POST /tasks	201	Task added
Update	PUT /tasks/1	200	Values changed
Delete	DELETE /tasks/1	200	Task removed
Verify delete	GET /tasks/1	404	Deletion confirmed

21. Troubleshooting

Symptom	Likely cause	Fix
uvicorn not recognized	Environment inactive or package missing	Activate .venv and install fastapi uvicorn.
Error loading ASGI app	main:app does not match file/object	Confirm main.py contains app = FastAPI().
Unexpected 404	Wrong path or HTTP method	Check registered routes in /docs.
POST has no body field	Body parameter not declared	Use parameter: dict = Body(...).
/tasks/abc fails	task_id expects int	This is FastAPI validation working.
Data disappears after restart	Data is only in memory	Expected; database comes later.

22. Pydantic - Introduced Last

Plain dictionaries are useful for learning, but they do not describe a strong request schema. Pydantic lets you declare the expected data shape and lets FastAPI validate it automatically.

PYTHON

```
from pydantic import BaseModel

class TaskCreate(BaseModel):
    title: str
```

Line / Code	What it does	Why it matters
from pydantic import BaseModel	Imports the model base class.	FastAPI integrates with these models.
class TaskCreate(BaseModel):	Defines a request schema.	The API now has an explicit expected structure.
title: str	Declares a required string.	Invalid or missing title data can be rejected automatically.

The POST endpoint can then become:

PYTHON

```
@app.post("/tasks", status_code=201)
def create_task(task: TaskCreate):
    new_task = {
        "id": len(tasks) + 1,
        "title": task.title,
        "completed": False
    }
    tasks.append(new_task)
    return new_task
```

Line / Code	What it does	Why it matters
task: TaskCreate	Parses the request body through the model.	The endpoint receives structured validated data.
task.title	Reads the validated field.	Attribute access is clearer than arbitrary dictionary keys.

Why it comes last

Pydantic is easier to understand as an improvement to request-body structure after students already understand routes, bodies and CRUD.

23. Post-Class Mini Backend Project

Project: Simple Book Management API

Your task is to build a small backend API from scratch using only the FastAPI concepts covered in this lesson. Do not use a database yet, and do not use Pydantic for the main solution. The goal is to prove

that you understand FastAPI routes, HTTP methods, CRUD, path parameters, query parameters, request bodies, in-memory data, status codes and HTTPException.

PROJECT SCENARIO

A small training-centre library needs a basic backend service for keeping track of books. Staff should be able to create books, view books, view one book, update a book, delete a book, and optionally filter books by availability.

Task Requirements

1. Create a FastAPI application in a file named main.py.

2. Create an in-memory list named books with at least two starting books.

3. Each book should have: id, title, author and available.

4. GET /books must return all books.

5. GET /books/{book_id} must return one matching book.

6. GET /books/{book_id} must raise 404 when the ID does not exist.

7. GET /books must support an optional query parameter: available=true or available=false.

8. POST /books must accept a JSON body as a plain Python dictionary and create a new book.

9. POST /books must return status code 201.

10. PUT /books/{book_id} must update title, author and available.

11. DELETE /books/{book_id} must remove the book.

12. Use FastAPI /docs to test every endpoint.

Before Looking at the Answer

Try to complete the project yourself first. Use the CRUD mapping you learned: Create -> POST, Read -> GET, Update -> PUT, Delete -> DELETE. If you get stuck, check your route decorator, function parameters, and whether the resource ID actually exists in your list.

Worked Answer

PYTHON

```
from fastapi import FastAPI, Body, HTTPException

app = FastAPI(title="Book Management API")

books = [
    {"id": 1, "title": "Python Basics", "author": "A. Musa", "available": True},
    {"id": 2, "title": "Web Development", "author": "J. Bello", "available": False},
```

```
]

@app.get("/")
def home():
    return {"message": "Book Management API is running"}

@app.get("/books")
def get_books(available: bool | None = None):
    if available is None:
        return books

    return [book for book in books if book["available"] == available]

@app.get("/books/{book_id}")
def get_book(book_id: int):
    for book in books:
        if book["id"] == book_id:
            return book

    raise HTTPException(status_code=404, detail="Book not found")

@app.post("/books", status_code=201)
def create_book(book: dict = Body(...)):
    new_book = {
        "id": len(books) + 1,
        "title": book.get("title", "Untitled"),
        "author": book.get("author", "Unknown"),
        "available": book.get("available", True),
    }

    books.append(new_book)
    return new_book

@app.put("/books/{book_id}")
def update_book(book_id: int, book_data: dict = Body(...)):
    for book in books:
        if book["id"] == book_id:
            book["title"] = book_data.get("title", book["title"])
            book["author"] = book_data.get("author", book["author"])
            book["available"] = book_data.get("available", book["available"])
            return book

    raise HTTPException(status_code=404, detail="Book not found")

@app.delete("/books/{book_id}")
def delete_book(book_id: int):
    for book in books:
        if book["id"] == book_id:
            books.remove(book)
            return {"message": "Book deleted successfully"}

    raise HTTPException(status_code=404, detail="Book not found")
```

How the Answer Works

Imports: `FastAPI` creates the application, `Body` tells FastAPI to read JSON from the request body, and `HTTPException` lets the API return real HTTP errors such as 404.

Application object: `app = FastAPI(...)` creates the FastAPI application that Uvicorn will run.

In-memory data: `books` is only a Python list. It behaves like temporary storage and will reset when the server restarts.

Read all + filter: `GET /books` returns the full list when no query parameter is supplied. When `available=true` or `available=false` is supplied, the list comprehension filters the books.

Read one: `GET /books/{book_id}` uses a path parameter to identify one resource. The loop searches by ID and a missing ID produces 404.

Create: `POST /books` receives a JSON body as a Python dictionary, builds a new book, appends it to the list and returns 201 Created.

Update: `PUT /books/{book_id}` locates the book and changes its values. `dict.get()` allows existing values to remain when a field is not supplied.

Delete: `DELETE /books/{book_id}` removes the matching dictionary from the list and returns a confirmation message.

How to Test the Project

Request	Expected Result
GET /books	Expect all books.
GET /books/1	Expect book with ID 1.
GET /books/999	Expect 404 Book not found.
GET /books?available=true	Expect only available books.
POST /books	Send JSON containing title, author and available. Expect 201.
PUT /books/1	Send JSON with changed values. Expect updated book.
DELETE /books/1	Expect success message, then GET /books/1 should return 404.

Student Extension Challenge

After the required project works, add ONE of the following without looking at a solution first:

• Add GET /books?author=... to filter by author.

• Add PATCH /books/{book_id}/borrow that changes available to False.

• Add PATCH /books/{book_id}/return that changes available to True.

• Prevent deletion of a missing book by returning 404 (if not already implemented).

• Add a second resource of your choice, such as courses or students, with at least GET and POST routes.

IMPORTANT
Do not add Pydantic to this project until the FastAPI + CRUD version works correctly. Pydantic is the next improvement, not the starting point.

24. Self-Assessment

1. What job does FastAPI perform?
2. What job does Uvicorn perform?
3. What does @app.get("/tasks") mean?
4. Why can GET /tasks and POST /tasks share a path?
5. Explain CRUD without mentioning FastAPI.
6. Match CRUD operations to typical HTTP methods.
7. Difference between /tasks/7 and /tasks?completed=true?
8. Why does task_id: int matter?
9. What does Body(...) do?
10. Why use 201 after creation?
11. Why use 404 for a missing resource?
12. Describe POST /tasks from request to response.
13. Why does in-memory data disappear after restart?
14. What does Pydantic improve?
15. Why was Pydantic placed at the end?

25. Answer Key

1. FastAPI defines API routes, extracts request data, runs endpoint functions and builds responses.
2. Uvicorn listens for HTTP traffic and runs the FastAPI application.
3. It registers GET /tasks and connects it to the following function.
4. Method + path together identify an endpoint; GET and POST are different operations.
5. CRUD means Create, Read, Update and Delete operations on application data.

6. Create -> POST; Read -> GET; Update -> usually PUT/PATCH; Delete -> DELETE.

7. /tasks/7 uses a path parameter to identify one resource; ?completed=true is a query filter.

8. FastAPI validates and converts the path value to int.

9. Body(...) tells FastAPI to read that function argument from the request body.

10. 201 Created specifically says a resource was created.

11. 404 communicates that the requested resource does not exist.

12. Uvicorn receives the request, FastAPI matches the route, extracts the body, Python code creates/appends data, FastAPI serializes the return value and sends the response.

13. The list exists only in process memory and is recreated when the program restarts.

14. Pydantic adds explicit schemas, type validation and better generated documentation.

15. Students first need to understand routes, bodies and CRUD so Pydantic is seen as an improvement rather than unexplained syntax.

26. Completion Checklist

☐ I can explain FastAPI practically.

☐ I can explain FastAPI vs Uvicorn.

☐ I can start the server.

☐ I can use /docs.

☐ I can create GET routes.

☐ I can use path parameters.

☐ I can use query parameters.

☐ I can receive a JSON body as a dict.

☐ I can explain CRUD clearly.

☐ I can implement Create, Read, Update and Delete.

☐ I can return 404 with HTTPException.

☐ I understand temporary in-memory data.

☐ I can explain what Pydantic adds.